

Univerzitet u Novom Sadu
Fakultet tehnickih nauka

Coworking Booking System

Projektna dokumentacija

Predmet: Integracije - IROIT

Student: Andjela Lukic

Broj indeksa: IT62/2022

GitHub: github.com/IROIT-2025/iroit-grupa-3-4-andjelalukicc

Akadska godina: 2025/2026.

Sadržaj

1. Uvod	3
2. Arhitektura sistema	3
2.1 Pregled sistema	3
2.2 Uloge mikroservisa	4
2.3 Logovanje i pracenje zahteva	5
3. Tipovi komunikacije izmedju servisa	5
3.1 Sinhrona komunikacija - REST API	5
3.2 Asinhrona komunikacija - RabbitMQ	6
3.3 Reaktivna komunikacija - SSE	6
4. Git workflow i verzionisanje	7
5. Testiranje	7
5.1 Unit testovi	7
5.2 Integracioni testovi (E2E)	8
6. Kontejnerizacija - Docker	9
6.1 Dockerfile	9
6.2 Docker Compose orkestracija	9
7. CI pipeline - GitHub Actions	10
8. Staticka analiza koda - ESLint	11
9. Deployment	12
10. CD pipeline	12
11. Monitoring i observability	13
11.1 Prometheus - prikupljanje metrika	13
11.2 Grafana - vizualizacija	14
12. Zakljucak	14

1. Uvod

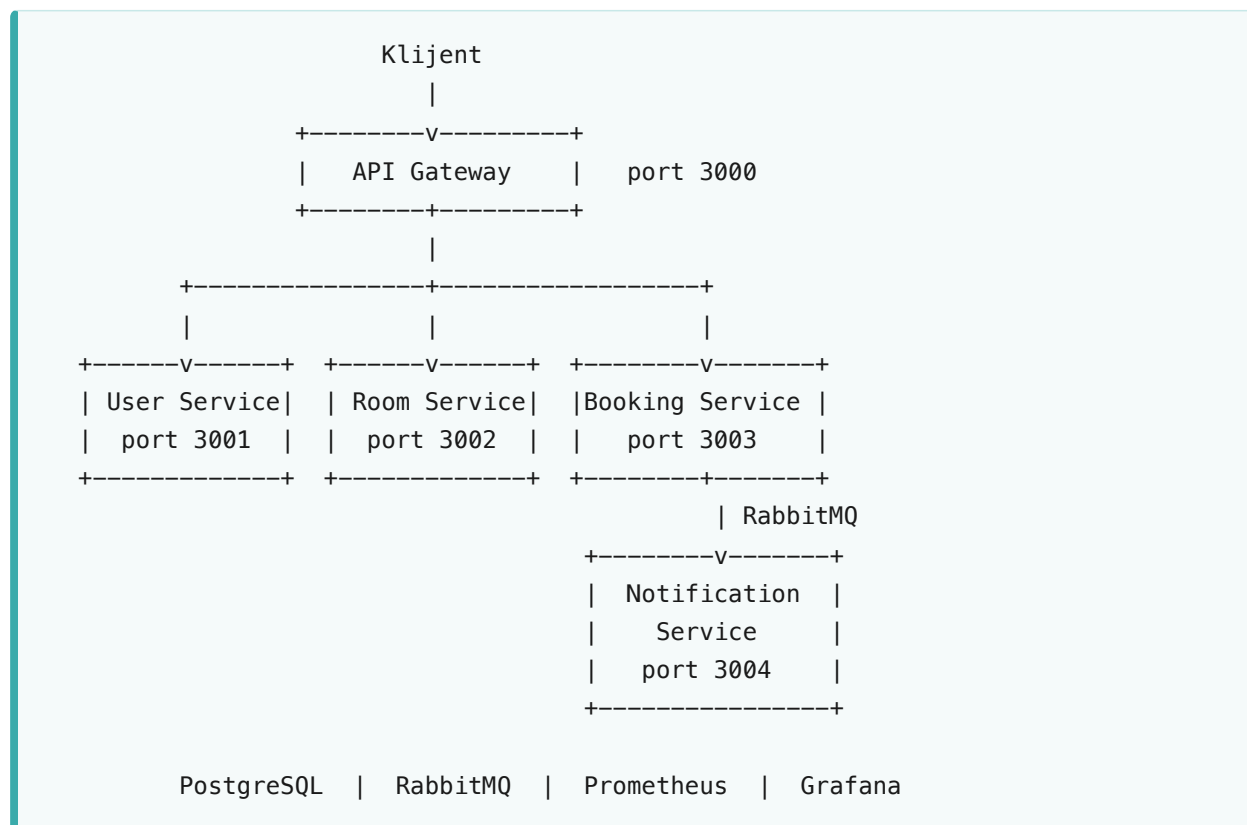
Tema ovog projekta je izgradnja sistema za rezervaciju prostorija u coworking prostoru, realizovana kroz mikroservisnu arhitekturu. Cilj projekta bio je da se u praksi primene ključni DevOps koncepti koji su obrađivani na predmetu Integracije: različiti tipovi komunikacije između servisa, automatizacija procesa razvoja putem CI/CD pipeline-a, kontejnerizacija, statička analiza koda i monitoring aplikacije.

Projekat je razvijen korištenjem Node.js frejm vorka NestJS i pokreće se lokalno putem Docker Compose-a. Sistem se sastoji od pet nezavisnih mikroservisa koji međusobno komuniciraju putem REST API-ja, message broker-a RabbitMQ i reaktivnih stream-ova (SSE). Svaki servis ima jasno definisanu odgovornost i sopstvene podatke u bazi.

2. Arhitektura sistema

2.1 Pregled sistema

Sistem je implementiran kao skup od pet nezavisnih mikroservisa. Klijent nikada ne komunicira direktno sa pojedinim servisima - sve zahteve prima API Gateway koji ih prosledjuje odgovarajućem servisu. Ovakav pristup, poznat kao Gateway pattern, omogućava da svaki servis može da se razvija, testira i skalira nezavisno od ostalih.



Slika 1 - Dijagram arhitekture sistema

2.2 Uloge mikroservisa

API Gateway (port 3000) - jedina tacka ulaska u sistem. Prima sve zahteve od klijenta, proverava JWT token na zasticenim rutama i prosledjuje zahtev odgovarajućem servisu.

Javne rute (registracija, login, lista prostorija) ne zahtevaju autentifikaciju. Gateway takodje dodaje `x-user-id` header u interne zahteve, kako bi servisi znali koji korisnik je ulogovan bez ponovnog dekodiranja tokena.

User Service (port 3001) - zaduzenje za upravljanje korisnicima. Podrzava registraciju, login i pregled profila. Lozinka se cuva u hashovanom obliku koristenjem bcrypt algoritma, a nakon uspesnog login-a klijentu se vraća JWT token koji vazi 24 sata.

Room Service (port 3002) - upravlja podacima o prostorijama. Pri prvom pokretanju automatski upisuje 8 prostorija u bazu: 2 meeting room-a i 6 phone booth-ova. Pored standardnih REST endpointa, implementira SSE stream koji klijentima salje obavestavanja o promeni dostupnosti u realnom vremenu.

Booking Service (port 3003) - najslozeniji deo sistema koji sadrzi svu poslovnu logiku vezanu za rezervacije. Pre kreiranja rezervacije sistem proverava: da li je prostorija slobodna u zeljenom terminu, da li korisnik vec ima rezervaciju koja se preklapa i da li je dostignut dnevni limit od tri rezervacije. Tek ako su svi uslovi ispunjeni, rezervacija se kreira i salje se event putem RabbitMQ-a.

Notification Service (port 3004) - radi kao hibridna aplikacija, istovremeno prima HTTP zahteve i slusa poruke sa RabbitMQ reda. Kada Booking Service posalje event `booking_created` ili `booking_cancelled`, ovaj servis kreira obavestenje i cuva ga u bazi. Korisnik moze da procita svoja obavestenja putem REST endpointa.

Servis	Port	Odgovornost
API Gateway	3000	Rutiranje, JWT autentifikacija, rate limiting, logovanje
User Service	3001	Registracija, login, upravljanje korisnicima
Room Service	3002	Upravljanje prostorijama, SSE dostupnost u realnom vremenu
Booking Service	3003	Kreiranje i otkazivanje rezervacija, poslovna logika
Notification Service	3004	Obavestenja korisnika putem RabbitMQ event-a
PostgreSQL	5432	Relaciona baza podataka
RabbitMQ	5672 / 15672	Message broker i management UI
Prometheus	9090	Prikupljanje metrika
Grafana	3100	Vizualizacija metrika i dashboardi

Tabela 1 - Pregled svih servisa sa portovima i ulogama

2.3 Logovanje i pracenje zahteva

Svaki mikroservis belezi logove za sve dolazece zahteve i izlazece odgovore. U API Gateway-u implementiran je globalni logger middleware koji za svaki zahtev belezi HTTP metod, putanju, status kod odgovora i vreme trajanja obrade. Ovakav pristup omogucava pracenje celokupnog toka zahteva kroz sistem i znacajno olaksava otklanjanje gresaka.

Pored logova na nivou zahteva, Booking Service i Notification Service belezu i klucne poslovne dogadjaje - kreiranje i otkazivanje rezervacija, kao i prijem RabbitMQ event-a. Metrike iz svakog servisa dostupne su putem `/metrics` endpointa i prikuplja ih Prometheus, sto u kombinaciji sa Grafana dashboardom pruza potpunu observability pokrivenost sistema.

3. Tipovi komunikacije izmedju servisa

3.1 Sinhrona komunikacija - REST API

REST API koristi se za komunikaciju izmedju API Gateway-a i svih internih servisa, kao i izmedju Booking Service-a i Room Service-a pri proveru dostupnosti prostorije. Radi se o sinhronoj komunikaciji - servis salje zahtev i ceka odgovor pre nego sto nastavi sa obradom. Ovaj tip komunikacije pogodan je za situacije u kojima je odgovor neophodan za dalju obradu, kao sto je provera slobodnih termina pre kreiranja rezervacije.

Svi endpointi su dokumentovani putem Swagger/OpenAPI i dostupni su na adresi `http://localhost:3000/api` nakon pokretanja aplikacije.

3.2 Asinhrona komunikacija - RabbitMQ

RabbitMQ message broker koristi se za komunikaciju izmedju Booking Service-a i Notification Service-a. Nakon kreiranja ili otkazivanja rezervacije, Booking Service objavljuje event na RabbitMQ red, a Notification Service ga nezavisno preuzima i obrađuje. Ova komunikacija je asinhrona - Booking Service ne ceka odgovor i ne mora da zna da Notification Service uopste postoji.

Ovakav pristup smanjuje medjuzavisnost servisa i povecava otpornost sistema na greske: cak i ako Notification Service nije dostupan, rezervacija ce biti uspesno kreirana, a obavestenje ce biti isporuceno cim servis ponovo postane aktivan.

```
// Booking Service - slanje eventa nakon uspesne rezervacije
this.rabbitClient.emit('booking_created', {
  userId:    booking.userId,
  roomId:    booking.roomId,
  date:      booking.date,
  startTime: booking.startTime,
  endTime:   booking.endTime,
});
```

```
// Notification Service - prijem eventa sa RabbitMQ reda
```

```
@EventPattern('booking_created')
async handleBookingCreated(data: BookingCreatedDto) {
    await this.notificationsService.create(data);
}
```

Isecak koda 1 - Asinhrona komunikacija putem RabbitMQ

3.3 Reaktivna komunikacija - SSE (Server-Sent Events)

Server-Sent Events implementiran je u Room Service-u i omogućava klijentu da prima obavestjenja o promeni dostupnosti prostorija u realnom vremenu, bez potrebe da periodicno salje zahteve serveru. Klijent otvori jednu HTTP konekciju ka endpointu /rooms/availability/stream i server mu salje podatke cim se stanje promeni. Ovaj pristup je znatno efikasniji od klasicnog pollinga jer smanjuje broj nepotrebnih zahteva i cini sistem reaktivnim u distribuciji podataka.

Tip komunikacije	Gde se koristi	Karakteristika
REST API (sinhrona)	Gateway prema svim servisima, Booking prema Room Service-u	Klijent ceka odgovor pre nastavka
RabbitMQ (asinhrona)	Booking Service prema Notification Service-u	Fire-and-forget, bez cekanja odgovora
SSE (reaktivna)	Room Service prema klijentu	Server inicira slanje podataka u realnom vremenu

Tabela 2 - Pregled tipova komunikacije u sistemu

4. Git workflow i verzionisanje

Za verzionisanje koda koriscena je GitHub platforma, a razvojni proces je organizovan po principu feature branch workflow-a. Svaka znacajna izmena razvijana je na zasebnoj feature grani i spojena u glavnu granu isključivo putem Pull Request-a, nakon sto su svi automatski CI proverili prosli uspesno. Ovakav pristup garantuje da main grana uvek sadrzi stabilan i testiran kod.

Pull Request	Opis izmene	Status
PR #1	Initialize NestJS microservices architecture	Merged
PR #2	Add health check endpoints to all microservices	Merged
PR #3	Add Swagger/OpenAPI documentation	Merged
PR #4	Add E2E integration tests - 34 testova	Merged
PR #5	Add CORS support and request logging	Merged
PR #6	Add global validation pipe and exception filter	Merged

Tabela 3 - Istorija Pull Request-ova na GitHub repozitorijumu

Za svaki Pull Request automatski je pokrenuto CI testiranje koje je moralo biti uspesno pre mergea. Na taj nacin su potencijalne greske otkrivane rano, pre nego sto bi dosle u glavnu granu. Repozitorijum je dostupan na adresi: github.com/IR0IT-2025/iroit-grupa-3-4-andjelalukicc

5. Testiranje

Testiranje je uvedeno paralelno sa razvojem aplikacije i sprovodi se na dva nivoa: unit testovi koji pokrivaju poslovnu logiku u izolaciji, i end-to-end testovi koji proveravaju ispravnost kompletnih HTTP tokova. Svi testovi su integrisani u CI pipeline i moraju prolaziti pri svakom push-u.

5.1 Unit testovi

Unit testovi napisani su za svih pet servisa koristenjem Jest frej mvorka koji dolazi ugraden u NestJS. Umesto pravih baza podataka i message broker-a, koriste se mock objekti koji simuliraju njihovo ponasanje. Na taj nacin testovi su brzi, pouzdani i ne zahtevaju pokrenute spoljne servise.

Posebna paznja posvecena je testiranju rubnih slucajeva u Booking Service-u, koji sadrzi najslozeniju poslovnu logiku u sistemu.

Servis	Broj testova	Sta se testira
User Service	7	Registracija, login, dupli email, pretraga korisnika, greske
Booking Service	14	Kreiranje, otkazivanje, preklapanje termina, dnevni limit, validacije

Servis	Broj testova	Sta se testira
Room Service	8	Lista prostorija, filtriranje po tipu, seed logika, SSE stream
Notification Service	6	Kreiranje obavestenja, obrada RabbitMQ eventa, citanje
API Gateway	5	Health endpoint, proxy rutiranje
Ukupno	40	Svi testovi prolaze

Tabela 4 - Pregled unit testova po servisima

5.2 Integracioni testovi (E2E)

End-to-end testovi testiraju kompletne HTTP zahteve prema stvarnim endpointima sa pravom bazom podataka. Za razliku od unit testova koji testiraju logiku u izolaciji, E2E testovi pokrecu ceo servis i proveravaju da sve komponente zajedno funkcionisu ispravno. U Booking Service-u se takodje proverava da li je RabbitMQ event ispravno emitovan nakon kreiranja rezervacije.

Servis	Broj testova	Sta se testira
Booking Service	20	HTTP endpointi za kreiranje, pregled i otkazivanje rezervacija, RabbitMQ event verifikacija
User Service	14	Registracija, login, JWT validacija, zasticeni profil endpoint
Ukupno	34	Svi testovi prolaze

Tabela 5 - Pregled E2E testova po servisima

Ukupan broj testova u projektu iznosi **74** (40 unit + 34 E2E), svi prolaze bez gresaka.

6. Kontejnerizacija - Docker

6.1 Dockerfile

Svaki mikroservis ima sopstveni Dockerfile koji koristi multi-stage build pristup. U prvoj fazi (builder) kompajlira se TypeScript kod u JavaScript, a u drugoj fazi (production) u finalnu sliku kopira se samo kompajlirani kod i produkcione zavisnosti. Razvojni alati i izvorni kod ne ulaze u finalnu sliku, sto je cini manjom i bezbednijom. Pored Dockerfile-a, svaki servis ima i `.dockerignore` fajl koji iz Docker konteksta iskljucuje `node_modules`, `dist` folder i razvojne konfiguracione fajlove.

```
# Faza 1 - kompajliranje TypeScript koda
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Faza 2 - produkciona slika (samo dist i produkcione zavisnosti)
FROM node:22-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/main"]
```

Isecak koda 2 - Multi-stage Dockerfile (primer za jedan servis)

6.2 Docker Compose orkestracija

Svi servisi definisani su i orkestrirani u jednom `docker-compose.yml` fajlu koji pokreće ukupno devet kontejnera: pet NestJS mikroservisa, PostgreSQL, RabbitMQ, Prometheus i Grafana. Svi kontejneri nalaze se u zajedничој Docker mreži `coworking-network`, što im omogućava međusobnu komunikaciju putem naziva servisa umesto IP adresa.

Za PostgreSQL i RabbitMQ definisani su `healthcheck`-ovi koji proveravaju da li je servis spreman pre nego što se NestJS servisi počnu pokretati. Environment varijable (konekcija na bazu, RabbitMQ URL, JWT tajni ključ) definisane su direktno u `docker-compose.yml` fajlu. Za bazu podataka definisan je `named volume postgres_data` koji obezbeđuje perzistenciju podataka - podaci u bazi ostaju sacuvani i nakon restarta kontejnera.

```

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: coworking
    volumes:
      - postgres_data:/var/lib/postgresql/data # perzistencija podataka
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5

  booking-service:
    build: ./booking-service
    environment:
      DATABASE_URL: postgresql://postgres:postgres@postgres:5432/coworking
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
      JWT_SECRET: your-secret-key
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    networks:
      - coworking-network

volumes:
  postgres_data: # named volume za perzistenciju baze

networks:
  coworking-network:

```

Isecak koda 3 - Docker Compose konfiguracija sa volumeima i healthcheck-ovima

Aplikacija se pokrece jednom komandom: `docker compose up -d`. Nakon pokretanja, svih devet kontejnera su u statusu `healthy` i sistem je spreman za korištenje.

7. CI pipeline - GitHub Actions

Svaki push na GitHub i svaki otvoreni Pull Request automatski pokreceu CI (Continuous Integration) pipeline definisan u fajlu `.github/workflows/ci.yml`. Cilj CI pipeline-a je da se automatski proveri ispravnost koda pre nego sto bude spojen u glavnu granu. Ako bilo koji korak u pipeline-u ne prodje, Pull Request ne moze biti spojen u `main` granu.

Pipeline koristi matrix strategiju, sto znaci da se isti skup koraka izvrsava paralelno za svih pet servisa. Na taj nacin celokupni proces traje znatno krace nego kada bi se servisi obradivali jedan po jedan.

```

strategy:
  matrix:

```

```

    service: [api-gateway, user-service, room-service,
              booking-service, notification-service]

steps:
  - name: Checkout koda
    uses: actions/checkout@v4

  - name: Setup Node.js 22
    uses: actions/setup-node@v4
    with:
      node-version: '22'

  - name: Cache node_modules (ubrzava pipeline)
    uses: actions/cache@v4

  - name: Instalacija zavisnosti
    run: npm install

  - name: ESLint - staticka analiza koda
    run: npx eslint src/

  - name: Build - kompilacija TypeScript-a
    run: npm run build

  - name: Pokretanje unit testova
    run: npx jest --passWithNoTests

  - name: Build Docker image (kreiranje build artefakta)
    run: docker build -t coworking-${{ matrix.service }} .

```

Isecak koda 4 - CI pipeline konfiguracija (GitHub Actions)

Pipeline se sastoji od dva uzastopna job-a. U prvom job-u (build-and-test) vrši se instalacija zavisnosti, staticka analiza, kompilacija i pokretanje testova za svih pet servisa paralelno. Drugi job (docker-build) pokrece se tek nakon sto prvi uspesno završi i kreira Docker slike za sve servise, cime se verifikuje da je build artefakt ispravan.

8. Staticka analiza koda - ESLint

Staticka analiza koda implementirana je korištenjem ESLint alata, koji je konfigurisan za svih pet servisa. Koriscen je flat config format (`eslint.config.mjs`), koji je standard od ESLint v9. Integriran je TypeScript ESLint plugin zajedno sa Prettier alatom za formatiranje koda.

Analiza pokriva code smells, potencijalne bugove i sigurnosne probleme u kodu. Pravila su prilagodjena specifičnostima projekta - u API Gateway-u su isključena `no-unsafe-*` pravila jer proxy pattern inherentno zahteva rad sa any tipovima pri prosledjivanju zahteva. Za sve ostale servise primenjena su standardna TypeScript ESLint pravila koja pokrivaju neiskoriscene promenljive, potencijalne null-pointer probleme i losije prakse u pisanju koda.

Rezultat staticke analize: **0 gresaka u svih 5 servisa.**

ESLint se izvrsava kao prvi korak u CI pipeline-u, pre kompilacije i testova. Na taj nacin, kod koji ne prolazi analizu ne moze biti ni kompajliran ni testiran.

9. Deployment

Aplikacija se deployi lokalno putem Docker Compose-a, sto je u skladu sa zahtevom specifikacije koji dozvoljava potpuno lokalno kontejnerizovano okruzenje. Ceo razvojni stack je kontejnerizovan - ne postoji nista sto se pokrece van Docker okruzenja.

Deployment obuhvata sve kljucne komponente: pet NestJS mikroservisa, PostgreSQL bazu podataka, RabbitMQ message broker, Prometheus i Grafana za monitoring. Environment varijable su konfigurisane u `docker-compose.yml` fajlu. API Gateway implementira health endpoint na adresi `/health` koji vraca status svakog internog servisa i koristi se za post-deploy proveru ispravnosti.

```
// Health endpoint odgovor - API Gateway
{
  "status": "ok",
  "services": {
    "user-service": "online",
    "room-service": "online",
    "booking-service": "online",
    "notification-service": "online"
  }
}
```

Isecak koda 5 - Odgovor health endpointa API Gateway-a

10. CD pipeline

CD (Continuous Delivery) pipeline definisan je u fajlu `.github/workflows/cd.yml` i pokrece se automatski kada se kod spoji u main granu, odnosno nakon merge-a Pull Request-a.

Pipeline se izvrsava u tri uzastopna koraka. U prvom koraku ponovo se pokrecu svi testovi za svih pet servisa - ovo je pre-deploy verifikacija koja osigurava da nijedan neispravan kod ne dodje u produkciju. U drugom koraku prave se Docker slike sa dva taga: latest koji uvek pokazuje na poslednju verziju i SHA hash trenutnog commit-a koji omogucava tacnu identifikaciju svake verzije. U trecem koraku vrsi se post-deploy provera ispravnosti - pokrecu se svih devet kontejnera i poziva se health endpoint API Gateway-a kako bi se potvrdilo da je ceo sistem operativan.

```
# Tagovanje Docker slike sa "latest" i SHA hash-om commit-a
docker build \
  -t coworking-${{ matrix.service }}:latest \
  -t coworking-${{ matrix.service }}:${{ github.sha }} \
  -f ${{ matrix.service }}/Dockerfile \
  ${{ matrix.service }}

# Post-deploy health check
docker compose up -d
sleep 30
curl -f http://localhost:3000/health
```

Isecak koda 6 - CD pipeline: tagovanje image-a i health check

11. Monitoring i observability

Za monitoring sistema koriscena je kombinacija Prometheus i Grafana alata. Prometheus prikuplja metrike od svakog mikroservisa, a Grafana ih vizualizuje u vidu interaktivnih dashboarda. Ovaj pristup omogućava pracenje zdravlja sistema u realnom vremenu i brzo reagovanje na eventualne probleme.

11.1 Prometheus - prikupljanje metrika

Svaki mikroservis eksponuje /metrics endpoint koristenjem prom-client biblioteke. Prometheus je konfigurisan da periodicki cita ovaj endpoint svakih 15 sekundi sa svih pet servisa i cuva prikupljene podatke u sopstvenoj bazi vremenskih serija.

Prikupljaju se sledece kategorije metrika: latency (vreme obrade zahteva), throughput (broj zahteva u jedinici vremena), error rate (udeo greskovnih odgovora 4xx i 5xx) i broj aktivnih konekcija po servisu.

```
# Prometheus konfiguracija - scraping svakih 15 sekundi
scrape_configs:
  - job_name: 'api-gateway'
    scrape_interval: 15s
    static_configs:
      - targets: ['api-gateway:3000']

  - job_name: 'user-service'
    scrape_interval: 15s
    static_configs:
      - targets: ['user-service:3001']

  - job_name: 'booking-service'
    scrape_interval: 15s
    static_configs:
      - targets: ['booking-service:3003']
```

Isecak koda 7 - Prometheus konfiguracija za scraping mikroservisa

11.2 Grafana - vizualizacija

Grafana je konfigurisana sa automatskim provisioning-om - datasource koji pokazuje na Prometheus i dashboard sa pet panela ucitavaju se automatski pri pokretanju sistema, bez potrebe za rucnom konfiguracijom. Ovo je realizovano putem YAML konfiguracionih fajlova u grafana/provisioning/ folderu koji su montiraju u kontejner putem Docker volume-a.

Panel	Tip prikaza	Sta prati
HTTP request rate	Line chart	Throughput - broj zahteva u sekundi po servisu
Latency percentili	Line chart	p50, p95, p99 - vreme obrade zahteva
Error rate	Stacked bars	Udeo 4xx i 5xx odgovora

Panel	Tip prikaza	Sta prati
Aktivne konekcije	Line chart	Broj aktivnih veza sa svakim servisom
Current request rate	Gauge	Trenutno opterecenje sistema

Tabela 6 - Pregled Grafana dashboard panela i sta svaki prati

Grafana je dostupna na <http://localhost:3100> sa podrazumevanim kredencijalima (admin/admin), a Prometheus na <http://localhost:9090>. RabbitMQ management UI dostupan je na <http://localhost:15672>.

12. Zaključak

U okviru ovog projekta implementiran je kompletan sistem za rezervaciju prostorija u coworking prostoru kao mikroservisna aplikacija. Sistem je razvijen u skladu sa svim zahtevima projektne specifikacije.

Implementirano je pet NestJS mikroservisa koji međusobno komuniciraju putem tri razlicita tipa komunikacije: sinhroni REST API, asinhroni RabbitMQ i reaktivni SSE stream. Celokupna infrastruktura je kontejnerizovana putem Docker Compose-a koji orkestrira devet kontejnera, ukljucujuci PostgreSQL bazu sa perzistentnim volume-om, RabbitMQ message broker i Prometheus/Grafana monitoring stack.

Razvojni proces organizovan je kroz Git feature branch workflow sa sest Pull Request-ova. Automatizovani CI/CD pipeline putem GitHub Actions obezbedjuje da svaki push i Pull Request produ kroz staticku analizu koda (ESLint - 0 gresaka), kompilaciju, 74 automatska testa (40 unit + 34 E2E) i Docker build verifikaciju pre spajanja u glavnu granu. CD pipeline se automatski pokrece na merge u main granu, kreira tagovan Docker image i vrsi post-deploy proveru ispravnosti putem health endpointa.

Monitoring sistema realizovan je putem Prometheus i Grafana alata koji prate klucne metrike svakog servisa: latency, throughput, error rate i broj aktivnih konekcija. Sve faze projekta su završene, svi testovi prolaze i aplikacija se uspesno pokrece lokalno.

Zahtev iz specifikacije	Implementacija
Minimalno 4 mikroservisa + API Gateway	5 mikroservisa + API Gateway
REST API komunikacija	Gateway prema svim servisima
Message Queue komunikacija	RabbitMQ: Booking prema Notification
Reaktivna komunikacija	SSE stream u Room Service-u
Logovanje metrika i trace-ova	prom-client + request logger middleware
Git workflow sa feature granama	6 Pull Request-ova, svi kroz feature grane
Unit testovi	40 testova, svi prolaze
E2E testovi	34 testa, svi prolaze
CI pipeline	GitHub Actions - build, lint, test, Docker
Docker kontejnerizacija	Multi-stage Dockerfile za svaki servis
Docker Compose orkestracija	9 kontejnera, mreza, volumes, healthcheck-ovi
Staticka analiza koda	ESLint - 0 gresaka u svih 5 servisa
Deployment sa health endpointom	Lokalni Docker, /health endpoint
CD pipeline	Automatski deploy na merge u main
Monitoring (latency, throughput, error rate)	Prometheus + Grafana dashboard

Tabela 7 - Pregled ispunjenih zahteva iz projektne specifikacije

Student: Andjela Lukic, IT62/2022

Predmet: Integracije (IROIT), FTN Novi Sad, 2025/2026.

GitHub: github.com/IROIT-2025/iroit-grupa-3-4-andjelalukicc